

הרצת תהליכים
ותהליכונים

מערכות הפעלה
לינוקס

הרצת תהליכונים

מבוא ל-WinAPI

מערכת ההפעלה Windows כוללת ממשק, שמאפשר למתכנתים לקרוא לפונקציות שונות של מערכת ההפעלה. מדוע להשתמש בממשק זה? יש לכך שתי סיבות.

הסיבה הראשונה, היא חיסכון בעבודה. לדוגמה, במקום לכתוב פונקציה שפותחת קובץ לקריאה, או בודקת ערך ברגיסטרי, אפשר להשתמש בפונקציות המוכנות CreateFileA ו-RegQueryValueEx.

הסיבה השניה היא שאין (כמעט) ברירה אחרת. כפי שלמדנו, עקב מעגלי האבטחה שלמערכת ההפעלה, אפליקציה שרצה ב-userland לא יכולה לגשת באופן חופשי לחומרה, כגון הדיסק הקשיח או הזיכרון. הכל חייב לעבור דרך הקרנל של מערכת ההפעלה. ה-API, קיצור של Application Programming Interface, הוא דרך נפוצה לאפשר למתכנתים גישה לתוכנה שגורם אחר פיתח, וספציפית WinAPI היא הדרך שבה מתכנתי מערכת ההפעלה Windows מאפשרים לתוכנה חיצונית לפנות לקרנל ולבקש ממנו שירותים שונים.

הרצת תהליכונים

הממשק של WinAPI מתועד כך שכל מתכנת יכול להכנס ולבדוק אילו פונקציות קיימות וכיצד יש לקרוא להן. כאן חשוב להזכיר שהפונקציות המוגדרות ב-WinAPI אינן פונות ישירות אל הקרנל אלא קוראות קודם כל לפונקציות מתוך ntdll – זהו קוד נוסף של מיקרוסופט, שגם הוא רץ ב-Userland ושם נמצאות הפונקציות שקוראות ישירות אל ה-Kernel. ה-ntdll נקרא גם Native API והוא כולל פונקציות של מיקרוסופט שרובן מוכרות רק למיקרוסופט. תיאורטית אפשר לקרוא מתוך הקוד לפונקציות של ntdll, לדוגמה במקום לקרוא ל-WriteFile מתוך WinAPI אפשר לקרוא ל-NtWriteFile מתוך ה-NativeAPI, אך הקושי הוא שבמרבית המקרים הפונקציות הללו לא יהיו מתועדות. מיקרוסופט גם אינה מתחייבת לשמור על הפונקציות והממשקים של ה-Native API שלה, רק של WinAPI.

כדי להפעיל את הפונקציות של WinAPI יש צורך לבצע include לקובץ windows.h. כדי לחסוך זמן קימפול מומלץ לבצע את ה-include בתוך הקובץ pch.h וכך להנות מהיתרון של pre-compilation. המשמעות היא שבפעם הראשונה מבוצע תהליך קימפול ארוך יותר אך בפעמים הבאות הקימפול מתקצר. זו פרקטיקה טובה עבור קבצי header גדולים שאינם משתנים לעיתים קרובות, ובפרט windows.h.

הרצת תהליכונים

טיפוסי משתנים בסיסיים

כאשר תכנתנו ב-C הכרנו אוסף של טיפוסי משתנים. `char`, `float`, `int` ועוד. ב-`windows.h` יש שמות אחרים של טיפוסי משתנים. לדוגמה `BYTE` הוא משתנה בגודל 8 ביט, `PCHAR` הוא מצביע ל-`char`. האם אנחנו חייבים להשתמש בטיפוסי המשתנים של `windows.h`? לא חייבים, אפשר לבחור אם להשתמש בהגדרות של `windows.h` או בהגדרות המוכרות משפת C. למעשה, הדברים זהים לחלוטין, מכיוון שכל טיפוסי המשתנים של `windows.h` הם פשוט קריאה לטיפוסים ב-C בשמות אחרים, באמצעות ההנחיה `typedef`. כך לדוגמה:

```
typedef CHAR    *PCHAR;
```

בעוד `CHAR` עצמו מוגדר כך:

```
typedef char    CHAR;
```

כך שלמעשה `PCHAR` הוא בעצם-

```
typedef char    *PCHAR;
```

באופן דומה מוגדרים כל יתר הטיפוסים של `windows.h`.

למרות שלא חייבים להשתמש בטיפוסי משתנים אלו, מומלץ לעבוד איתם מכיוון שכל הפונקציות של WinAPI מתועדות עם טיפוסי משתנים אלו. בלי לכל הפחות להכיר אותם, יהיה מאד קשה לקמפל את הקוד ללא שגיאות.

טיפוסי משתנים בסיסיים מרכזיים ב-windows.h:

```
typedef int          BOOL;
typedef unsigned char BYTE;
typedef char         CHAR;
typedef unsigned long DWORD;
typedef float        FLOAT;
typedef int          INT;
```

המידע המלא על טיפוסי משתנים אלו ועל כל יתר טיפוסי המשתנים של WinAPI נמצא בלינק הבא, באתר MSDN של מיקרוסופט:

<https://docs.microsoft.com/en-us/windows/desktop/winprog/windows-data-types>

הרצת תהליכונים

מצביעים

ל-WinAPI יש טיפוסים משתנים גם עבור מצביעים. לכל טיפוסים המשתנים שלעיל ניתן להוסיף P על מנת לציין pointer. לדוגמה PCHAR PWORD PINT וכו'.

בדרך כלל נראה את האות L כתובה לפני מצביעים. לדוגמה LPINT. מהי ה-L והאם היא הכרחית? עבור מעבדים ישנים של 16 ביט, הוגדרו מצביעים מסוג near ו-far, כאשר מצביע near שומר כתובת בגודל 16 ביט ואילו מצביע far שומר כתובת בגודל 32 ביט. עבור מעבדים של 32 ביט ומעלה אין הבדל עם או בלי L. בדרך כלל נראה את ה-L מקדימה את טיפוס המשתנה, מטעמי שמירה על תאימות תוכנה, אבל עבור קוד שאינו מיועד למעבדים עתיקים אין לכך משמעות.

הרצת תהליכונים

סוג מיוחד של מצביע הוא PVOID או LPVOID, כלומר מצביע על אובייקט שהוא VOID, כלום. מה ההגיון בלהצביע על אובייקט שהוא כלום? זהו בעצם מצביע שהוא כמו ג'וקר בחבילת קלפים- כמו שג'וקר יכול להיות כל קלף, לפי רצון השחקן, כך מצביע ל-VOID יכול להצביע על כל אובייקט שהמתכנת קובע עבורו. דבר זה מאפשר גמישות תכנותית, אך מאידך המתכנת צריך לזכור על איזה סוג של אובייקט המצביע מצביע בכל שלב. לדוגמה, אם המתכנת חושב שהמצביע מצביע על מחרוזת, אך למעשה הוא מצביע על INT, לא תהיה שגיאת קומפילציה אך עלולה להיגרם שגיאה בזמן ריצה.

קטע הקוד הבא ממחיש כיצד מצביע מסוג PVOID מקבל פעם אחת ערך של מצביע מסוג PINT ופעם אחרת מסוג PFLOAT, דבר שאי אפשר לעשות עם מצביע אחר.

```
PVOID joker = NULL;
INT num = 2;

PINT pnum = &num;

joker = pnum;

printf("Joker: %d\n", *(PINT)joker);


FLOAT pi = 3.1415;

PFLOAT pfl = &pi;

joker = pfl;

printf("Joker: %f\n", *(PFLOAT)joker);
```


הרצת תהליכונים

מחרוזות

ה-API תומך בשני סוגי מחרוזות. הסוג הראשון הוא תווי ASCII, בהם כל תו נשמר ב-char, 8 ביט. כיוון שתווי ASCII מוגבלים ל-256 ערכים ואינם מסוגלים לייצג את התווים שקיימים בכל השפות, פותחו תווי UNICODE, בהם כל תו נשמר ב-word, 16 ביט. כעת יש ברשותנו כ-65536 צירופי תווים אפשריים, וזה כבר מספיק טוב.

לדוגמה, קוד ה-ASCII של התו 'a' הוא 61 (בהקסדצימלי). ניתן לשמור אותו ב-8 ביט. ב-UNICODE הצירוף מורחב ל-16 ביט ולכן הקוד הוא 0061. לעומת זאת האות העברית 'א', לה אין קוד ASCII, יש ערך UNICODE של 0D5.

איך המחשב יודע לפרש את 0D5 בתור האות א' ולא בתור שני קודי ASCII, קוד 5 וקוד 0D? צריך להגדיר לו עם איזה סוג מחרוזת אנחנו עובדים. מחרוזות שהן מסוג UNICODE יכולו את האות W – קיצור של wide, ואילו מחרוזות ASCII לא יכולו W. דוגמאות לסוגי מחרוזות ומצביעים אליהן:

```
typedef CONST CHAR    *LPCSTR;  
typedef CONST WCHAR   *LPCWSTR;  
typedef WCHAR         *LPWSTR;
```

ואותן המחרוזות ללא תחילית ה-L (שכאמור במעבדי 32 ביט והלאה- המשמעות זהה):

```
typedef CONST CHAR    *PCSTR;  
typedef CONST WCHAR   *PCWSTR;  
typedef WCHAR         *PWSTR;
```

בחלק מהמקרים של מצביעים למחרוזות, נמצא את התו C, שמציין Const. כלומר אי אפשר לשנות את המחרוזת.

הדרך להגדיר מחרוזת מסוג W היא לכתוב לפני המחרוזת את התו L. לדוגמה:

```
LPCWSTR my_str = L"Hello";
```

במקרה זה יש צורך להגדיר C מכיוון שזו מחרוזת קבועה. כלומר my_str מצביע לאזור בזכרון שם נשמר Hello כקבוע. אם, לעומת זאת, אנחנו רוצים להגדיר מצביע למחרוזת שיכולה להשתנות:

```
WCHAR my_str [6] = L"Hello";
```

```
LPWSTR pstr = my_str;
```

שימו לב שיש צורך להגדיר מחרוזת בגודל 6 איברים, למרות שהמילה "Hello" היא בת 5 תווים, מכיוון שבסוף המחרוזת צריך לשמור מקום לתו שמציין סיום מחרוזת. תו זה מיוצג על ידי קוד הקסדצימלי 00, כך שבכל סוף מחרוזת נמצא קוד 00. תו סיום המחרוזת מאפשר לבצע פעולות שונות על מחרוזות, כגון לבדוק אורך של מחרוזת ולהדפיס מחרוזת. ללא תו סיום מחרוזת, הפונקציה שבודקת את אורך המחרוזת לא תדע היכן להפסיק את הספירה ופונקציית ההדפסה לא תדע היכן לעצור את ההדפסה.

כפי שראינו, ישנם תווים שה-Unicode שלהם כולל את הקוד 00. לדוגמה התו 'a' הוא 0x0061. במילים אחרות, אם נשלח מחרוזת Unicode לפונקציה שבודקת אורך מחרוזת או לפונקציה שמדפיסה מחרוזת, אנחנו עלולים לקבל תוצאה לא תקינה. לכן יש סט מיוחד של פונקציות, שמטפלות במחרוזות מסוג Unicode.

לדוגמה, במקום printf שמתאימה לקוד ASCII, הפונקציה wprintf מקבלת מחרוזת Unicode. לדוגמה, במקום strlen שמתאימה ל-ASCII, הפונקציה wcslen מוצאת אורך של מחרוזת Unicode.

ומה אם אנחנו רוצים לכתוב קוד שיתאים הן ל-Unicode והן ל-ASCII?

במקרה זה במקום האות W נכתוב את האות T. המנגנון עובד כך: כאשר מוגדרת מחרוזת עם T, הקומפיילר בודק אם יש הגדרה של `define UNICODE`. אם לא- נעשה שימוש בהגדרות של ASCII, ולהיפך. כך נראית הגדרה של טיפוס מחרוזת PTSTR:

```
#ifdef UNICODE
    typedef LPWSTR PTSTR;
#else
    typedef LPSTR PTSTR;
#endif
```

כפי שרואים, אם מוגדר Unicode אז הטיפוס הינו בעצם LPWSTR, אחרת- LPSTR.

הרצת תהליכונים

פונקציות עם סיומת A לעומת סיומת W

לאחר שהבנו את ההבדלים בין קוד ASCII לבין UNICODE, אפשר להבין את ההבדל בין גרסאות שונות שיש לפעמים לאותה פונקציה. עבור פונקציות שמקבלות מחרוזת בתור אחד הפרמטרים, יש חשיבות לדעת בדיוק את הפורמט של המחרוזת, כדי למנוע טעויות. פונקציות שמסתיימות ב-A יצפו לקבל מחרוזת ASCII, פונקציות שמסתיימות ב-W יצפו לקבל מחרוזת UNICODE.

לדוגמה CreateFile, גרסה אחת היא CreateFileA ואילו גרסה אחרת היא CreateFileW. חפשו את ההבדל בפרמטרים שלהן:

HANDLE CreateFileA(

LPCSTR

lpFileName,

DWORD

dwDesiredAccess,

DWORD

dwShareMode,

LPSECURITY_ATTRIBUTES lpSecurityAttributes,

DWORD

dwCreationDisposition,

DWORD

dwFlagsAndAttributes,

HANDLE

hTemplateFile

);

HANDLE CreateFileW(

LPCWSTR

lpFileName,

DWORD

dwDesiredAccess,

DWORD

dwShareMode,

LPSECURITY_ATTRIBUTES lpSecurityAttributes,

DWORD

dwCreationDisposition

DWORD

dwFlagsAndAttributes,

HANDLE

hTemplateFile

);

כפי שאתם רואים, כל ההבדל הוא תו אחד: בפונקציה CreateFileA הפרמטר הראשון הוא מסוג LPCSTR, ואילו בפונקציה CreateFileW הפרמטר הראשון הוא מסוג LPCWSTR. הפרמטר הזה הוא מצביע על שם הקובץ אותו מבקשים ליצור.

קריאה לפונקציות של WinAPI

התיעוד של כל הפונקציות נמצא באתר MSDN, קיצור של Microsoft Developers Network. אם אנחנו יודעים מה שם הפונקציה שאנחנו מחפשים, ניתן לגל msdn ושם הפונקציה, לדוגמה "msdn readfile", וצפוי שהתוצאה העליונה תהיה הדף ב-MSDN. נבצע קריאה מודרכת בעמוד של MSDN ReadFile. מצד שמאל אפשר לראות את כל שמות הפונקציות מסודרות לפי הסדר המילוני שלהן.

LockFileEx function

QueryDosDeviceW function

ReadFile function

ReadFileEx function

ReadFileScatter function

נראה של-ReadFile יש וריאציה שנקראת ReadFileEx. לא נעסוק כרגע בוריאציה הזו, רק חשוב לשים לב שכדאי לעבור על וריאציות של פונקציות שאנחנו מחפשים, לעיתים נמצא פונקציה יותר מתאימה ממה שחיפשנו.

החלק המרכזי של ReadFile כולל הסבר קצר על מה שהפונקציה מבצעת ומיד לאחריו את הפרמטרים שהיא מקבלת ומחזירה:

Syntax

```
BOOL ReadFile(  
    HANDLE          hFile,  
    LPVOID          lpBuffer,  
    DWORD           nNumberOfBytesToRead,  
    LPDWORD         lpNumberOfBytesRead,  
    LPOVERLAPPED    lpOverlapped  
);
```

לאחר מכן יש הסבר על כל פרמטר. לדוגמה:

nNumberOfBytesToRead

The maximum number of bytes to be read.

```
BOOL ReadFile(  
    HANDLE          hFile,  
    LPVOID          lpBuffer,  
    DWORD           nNumberOfBytesToRead,  
    LPDWORD         lpNumberOfBytesRead,  
    LPOVERLAPPED    lpOverlapped  
);
```

לפונקציה ReadFile יש כמה פרמטרים שברור שהיא חייבת לקבל. Handle לקובץ, מצביע לזיכרון שאליו אנחנו רוצים שהמידע מהקובץ יועתק, וכמובן כמה בתים להעתיק. אלו שלושת הפרמטרים הראשונים. לגבי שני הפרמטרים הנוספים יש לנו שתי אפשרויות. הראשונה היא להזין שם ערכים מתאימים, אחרי שבדקנו מה הפרמטרים הללו אומרים, או לנצל את האפשרות שכתובה בתיעוד ולהזין פשוט NULL.

לאחר פירוט הפרמטרים, מוסבר ערך החזרה של הפונקציה. כפי שאפשר לראות מה-Syntax שלה, ערך החזרה הוא BOOL, כלומר יש רק שתי אפשרויות- 0 או 1. אנחנו עדיין צריכים הבהרה מה מציין 0 ומה מציין

Return Value

If the function succeeds, the return value is nonzero (**TRUE**).

If the function fails, or is completing asynchronously, the return value is zero (**FALSE**). To get extended error information, call the [GetLastError](#) function.

אם יש בעיה והפונקציה נכשלה, בשלב ראשון כדאי לבדוק את קוד השגיאה. קוד השגיאה יכול להסביר לנו מה היתה הבעיה הספציפית שגרמה לפונקציה לחזור עם FALSE. כדי לבצע זאת, כל מה שאנחנו צריכים לעשות זה לקרוא בקוד שלנו ל-GetLastError. לדוגמה:

```
HANDLE    hfile;

INT        num_bytes = NUM_BYTES;

CHAR       data[NUM_BYTES];

LPVOID     pBuffer = data;

BOOL result = ReadFile(hfile, pBuffer, num_bytes, NULL, NULL);

if (result == FALSE)
{
    printf("%d", GetLastError());
    return 0;
}
```

כעת יודפס ל-Console מספר כלשהו. כדי להבין מה המשמעות שלו ניגש לדף קודי השגיאה ב-MSDN:

<https://docs.microsoft.com/he-il/windows/desktop/Debug/system-error-codes>

הפונקציה CreateThread יוצרת Thread חדש, תחת ה-Process שמריץ את CreateThread. במילים אחרות, אם אנחנו מריצים את התוכנית Hello.exe והיא מפעילה את הפונקציה CreateThread, יוצר Thread חדש תחת ה-Process שנקרא Hello.exe.

בעת יצירת ה-Thread, אנחנו קובעים שני דברים. הראשון- מה הפונקציה הראשית שתרוץ ב-Thread. נניח שהקוד שלנו כולל את הפונקציות bla ו-stam. כאשר אנחנו יוצרים Thread חדש, אפשר לקבוע שהוא יריץ את הפונקציה bla או את הפונקציה stom. הדבר השני – אנחנו יכולים להעביר פרמטר (כן, פרמטר אחד) שיועבר לפונקציה הנבחרת. מה עושים אם רוצים להעביר לפונקציה מספר פרמטרים? כאשר נגיע לגשר, נעבור אותו. בינתיים אתם מוזמנים לחשוב על פתרון אפשרי להעברה של יותר מפרמטר אחד.

שלב א'- הגדרת הפונקציה הנקראת

בשלב זה נלמד כיצד להגדיר פונקציה באופן ש-CreateThread יכול לקרוא לה.

הפונקציה חייבת להיות מוגדרת כך:

```
DWORD WINAPI function_name(LPVOID lparam);
```

ההנחיה WINAPI מגדירה את ה-Calling Convention של הפונקציה. הבנה מהו Calling Convention לא הכרחית לטובת התכנות, אבל זהו ידע חשוב לטובת משימות אחרות, כגון קריאת קוד אסמבלי. בתמצית, הכוונה לדרך שבה פרמטרים מועברים לפונקציה, לדרך בה מוחזר ערך מהפונקציה ולדרך שבה המחסנית מנוקה מהפרמטרים שהועברו אליה. אפשר לקרוא פרטים נוספים בספר האסמבלי של המרכז לחינוך סייבר, בפרק אודות המחסנית ופרוצדורות.

להלן דוגמה לקוד שכולל פונקציה שמוגדרת לפי הדרישות של CreateThread. הקוד כולל העברה של

```
#include "pch.h"
```

פרמטר מסוג LPVOID ושימוש בו:

```
DWORD WINAPI bla(LPVOID lparam) {  
    for (INT i=1; i<=*(PINT)lparam; i++) {  
        printf("bla\n");  
    }  
    return 1;  
}
```

```
int main()  
{  
    DWORD count = 2;  
    LPVOID pcount = &count;  
    bla(pcount);  
}
```

בקוד זה העברנו פרמטר יחיד. כפי שרואים, הגדרת הפונקציה מחייבת שליחה של פרמטר יחיד. אך זה לא אומר שאי אפשר להעביר יותר מפרמטר אחד- נוכל לעקוף את המגבלה אם ניצור Struct שיכלול את כל הפרמטרים שלנו, ונעביר אותו.

```
#include "pch.h"

struct MyStruct
{
    INT a;
    INT b;
};

DWORD WINAPI bla(LPVOID lparam) {
    struct MyStruct *nums = (MyStruct*)lparam;
    INT count = nums->a + nums->b;
    for (INT i = 1; i <= count; i++) {
        printf("bla\n");
    }
    return 1;
}

int main()
{
    struct MyStruct *my_struct =
        (MyStruct*)malloc(sizeof(MyStruct));
    my_struct->a = 3;
    my_struct->b = 4;
    LPVOID pstruct = my_struct;
    bla(pstruct);
}
```

שלב ב' - קריאה ל- CreateThread

קיים על הפונקציה CreateThread תיעוד מקיף ב-MSDN. בכל פעם שאתם צריכים להשתמש בפונקציה כלשהי של WinAPI, פשוט חפשו בגוגל את שם הפונקציה ו-MSDN.

על סמך התיעוד ב-MSDN, הפונקציה CreateThread מקבלת שישה פרמטרים, כאשר מתוכם רק שני פרמטרים (השלישי והרביעי) חייבים להקבע על ידי המשתמש, ואילו ארבעת הנותרים יכולים להיות NULL או אפס.

אם כך, שני הפרמטרים היחידים שאנחנו צריכים לקבוע הם שם הפונקציה שה-Thread יריץ, והפרמטר שהפונקציה תקבל. כל מה שנותר לנו לעשות הוא למלא את הפרמטרים השונים לפי התבנית. מומלץ לשמור על ההערות לגבי משמעות כל פרמטר- הן עוזרות בתהליך הדיבוג. לדוגמה, כדי לוודא שלא התבלבלנו ושינינו את סדר הפרמטרים המועברים.

```
int main()
{
    struct MyStruct *my_struct =
        (MyStruct*)malloc(sizeof(MyStruct));
    my_struct->a = 3;
    my_struct->b = 4;
    LPVOID pstruct = my_struct;
    HANDLE hThread = CreateThread(
        NULL,          //default security attributes
        0,             //default stack size
        bla,           //thread function
        pstruct,       //thread param
        0,             //default creation flags
        NULL           //return thread identifier
    );
}
```

הסיכוי לכך שיודפס

משהו הוא אמנם קיים אבל קלוש. מיד נבין למה. נראה כאילו דבר לא מתרחש. האם זה מכיוון שיש לנו באג כלשהו בקוד? אולי לא נוצר Thread חדש? אולי יש בעיה עם ההדפסה מה-Thread החדש אל המסך? כדי לנסות להגיע לבד אל התשובה, הוסיפו לאחר הקריאה ל-CreateThread מספר שורות קוד שלא מבצעות כלום. לדוגמה:

```
for (INT i = 0; i < 100000; i++)  
    continue;
```

הפתעה! לפתע מודפס למסך הפלט של הפונקציה שקראנו לה (אם זה לא קורה, הריצו פעם נוספת או הגדילו את כמות הפעמים שהלולאה מתבצעת). חישבו- מדוע זה קורה? מדוע תוספת של לולאה שלא מבצעת כלום גורמת לכך שה-Thread שקראנו לו מדפיס פלט? כיצד הדברים קשורים? לאחר שחשבתם על כך, עיברו לחלק הבא.

נסביר מה התרחש וכיצד שורות הקוד שהוספנו השפיעו. זה התהליך שהתרחש לפני הוספת הלולאה המיותרת:

1. עם הרצת התוכנית נוצר Process חדש ובתוכו Thread ראשי. הפונקציה main שייכת לThread הראשי
2. ה-Thread הראשי ביצע פתיחה של Thread נוסף והעביר לו פונקציה ופרמטר
3. מיד לאחר פתיחת ה-Thread הנוסף, ה-Thread הראשי הגיע לסוף פונקציית ה-main. כאשר ה-Thread הראשי מגיע לסוף ה-main, ה-Process נסגר וסוגר איתו את כל ה-Thread'ים שעדיין פתוחים. במילים אחרות, ה-Thread הנוסף בכלל לא קיבל הזדמנות לרוץ

לאחר הוספת הלולאה ה"מיותרת", ה-Thread הראשי ממשיך לרוץ ולבצע את הלולאה. הוא אינו נסגר. הזמן בו ה-Thread הראשי ממשיך לרוץ, נותן הזדמנות ל-Scheduler של מערכת ההפעלה להקצות זמן מעבד ל-Thread הנוסף. ה-Thread הנוסף מסיים את העבודה שהגדרנו לו, ה-Scheduler חוזר ומקצה זמן מעבד ל-Thread הראשי עד גמר הריצה שלו וסגירה שלו ושל ה-Process.

למעשה, בזכות תוספת הלולאה המיותרת קיבלנו הצצה אל מנגנון התזמון של ה-Scheduler. אפשר לשחק עם המספרים בלולאה ולקבל הערכה לגבי הקצב שבו ה-Scheduler מחליף בין Thread'ים.

לאחר שהבנו את הבעיה, נתקן אותה. התיקון הוא פשוט ומתבצע על ידי הפונקציה WaitForSingleObject. פונקציה זו מקבלת אובייקט כלשהו- רשימת האובייקטים נמצאת בתעוד הפונקציה ב-MSDN והיא כוללת בין היתר אובייקטים מסוג Process, מסוג Mutex (סבלנות, ממש בקרוב נכיר) ומסוג Thread, כמובן. יחד עם האובייקט, מקבלת הפונקציה את כמות הזמן המקסימלי שהתכנית צריכה להמתין לאובייקט. לכן כל מה שאנחנו צריכים לעשות הוא להוסיף את הקריאה הבאה:

```
WaitForSingleObject(hThread, INFINITE);
```

בחלק זה נלמד להשתמש בפונקציה `CreateProcess`. לפני שנכנס לפרטים של הפרמטרים שלה, יש דבר כללי שצריך להבין: הפונקציה הזו מקבלת כפרמטר קובץ `exe` שיוּרָץ על ידי ה-`Process` החדש שיווצר. לכן נחלק את התרגיל שלנו לשני שלבים. בשלב הראשון ניצור קובץ `exe` שמבצע משהו קטן, ובשלב השני נלמד להשתמש ב-`CreateProcess` כדי להריץ אותו.

שלב א' – יצירת קובץ הרצה

למעשה ברגע שנקמפל את תוכנית ברירת המחדל שנוצרת על ידי Visual Studio כבר יש בידינו קובץ הרצה, אבל אנחנו נעשה לו שדרוג קטן על ידי הוספת ארגומנטים (תזכורת- ארגומנט הוא המידע שנשלח לתוך פונקציה, פרמטר הוא המשתנה המקומי בתוך הפונקציה שמקבל את הערך של הארגומנט) בכניסה ל-main.

```
int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Incorrect number of arguments\n");
        return 0;
    }
    int id = atoi(argv[1]);
    printf("Hello World! Process got argument %d\n", id);
    return 1;
}
```

```
int main(int argc, char *argv[])  
{  
    if (argc != 2) {  
        printf("Incorrect number of arguments\n");  
        return 0;  
    }  
    int id = atoi(argv[1]);  
    printf("Hello World! Process got argument %d\n", id);  
    return 1;  
}
```

הפרמטר הראשון `argc` הוא קיצור של `Argument Count`, כלומר מספר הארגומנטים שהפונקציה מקבלת.
הפרמטר השני `argv` הוא קיצור של `Argument Vector`, כלומר מערך של מצביעים לארגומנטים.

שלב ב' – יצירת תוכנית שקוראת לקובץ ההרצה שיצרנו

הפונקציה `CreateProcess` כוללת לא פחות מ-10 פרמטרים. התייעוד המלא נמצא כמובן ב-MSDN. עם זאת, מבין כל הפרמטרים יש למעשה רק פרמטר אחד שהוא חשוב לנו כרגע, כל יתר הפרמטרים יכולים לקבל ערכי ברירת מחדל. הפרמטר החשוב לנו הוא השני, כיוון שהוא מייצג את "שורת הפקודה" שמריצה את הפרוסס שאנחנו יוצרים. כלומר נרצה ליצור מחרוזת שתכיל את כל מה שייכנס ל-`argv` בפרוסס החדש שיווצר.

קטע הקוד הבא משמש ליצירת מחרוזת זו. שימו לב לשלבים:

א. הגדרת קובץ ההרצה והערך שנמסר כארגומנט בתור קבועים

ב. חישוב גודל הזיכרון שנצטרך להקצות, שהוא סכום המחרוזות שהגדרנו ועוד 2, לטובת תו רווח ותו

סיום מחרוזת `Null Termination`

ג. הקצאת מערך באמצעות `malloc`

ד. העתקת הערכים למערך באמצעות `sprintf_s`


```
#define EXE_FILENAME    "..\\..\\HelloWorld\\Debug\\HelloWorld.exe"
#define PROCESS_ARG     "4"

int main()
{
    // create argument string
    CHAR exe_filename[] = EXE_FILENAME;
    CHAR process_arg[] = PROCESS_ARG;
    INT size = strlen(exe_filename) + strlen(process_arg) + 2;
    size += 2 // space + null termination
    PCHAR param = (PCHAR)malloc(size * sizeof(CHAR));
    sprintf_s(param, size, "%s %s", exe_filename, process_arg);
```

שימו לב שהמחרוזת שיצרנו היא של תווי ASCII ולכן נקרא ל-CreateProcessA, גרסת ה-ASCII, ולא ל-CreateProcessW.

לטובת הקריאה יש סידור נוסף שאנחנו צריכים לבצע. יש צורך לאתחל את שני הפרמטרים האחרונים שמועברים ל-CreateProcessA.

- הערך הראשון הוא משתנה מטיפוס STRATUPINFOA. זהו מבנה נתונים שמכיל את הגדרות החלון שייפתח על ידי ה-Process בזמן הריצה, אם תחפשו עליו ב-MSDN תמצאו את מלוא השדות שהוא מכיל
- הערך השני הוא מבנה נתונים שקרוי Process Information. זהו מבנה פשוט למדי שכולל בסך הכל ארבעה שדות:
 - המספר המזהה של הפרוסס, ה-PID, שנקבע על ידי מערכת ההפעלה בזמן יצירת הפרוסס
 - המספר המזהה של ה-Main Thread, אותו Thread ראשון שמופעל עם יצירת ה-Process.
 - אובייקט Handle ל-Process
 - אובייקט Handle ל-Main Thread

בתהליך יצירת ה-Process אנחנו שולחים לפונקציה CreateProcess שני מבני נתונים מאופסים, שיתמלאו על ידי מערכת ההפעלה לפני שהיא תיצור את ה-Process.

בסיום הריצה, נשתמש בנתונים שנמצאים ב-Process Information על מנת לסגור את ה-Handle ל-Process שיצרנו ול-Main Thread שלו.

```
#define EXE_FILENAME    "..\\..\\HelloWorld\\Debug\\HelloWorld.exe"
```

```
#define PROCESS_ARG     "4"
```

```
int main()
```

```
{
```

```
    STARTUPINFOA si;
```

```
    PROCESS_INFORMATION pi;
```

```
    ZeroMemory(&si, sizeof(si));
```

```
    si.cb = sizeof(si);
```

```
    ZeroMemory(&pi, sizeof(pi));
```

```
// create argument string  
CHAR exe_filename[] = EXE_FILENAME;  
CHAR process_arg[] = PROCESS_ARG;  
INT size = strlen(exe_filename) + strlen(process_arg) + 2;  
PCHAR param = (PCHAR)malloc(size * sizeof(CHAR));  
sprintf_s(param, size, "%s %s", exe_filename, process_arg);
```

```
// Start the child process.  
CreateProcessA(NULL,  
param,      // Command line  
NULL,      // Process handle not inheritable  
NULL,      // Thread handle not inheritable  
FALSE,     // Set handle inheritance to FALSE  
0,         // No creation flags  
    NULL,      // Use parent's environment block  
NULL,      // Use parent's starting directory  
&si,       // Pointer to STARTUPINFO structure  
&pi);     // Pointer to PROCESS_INFORMATION structure  
CloseHandle(pi.hThread);  
CloseHandle(pi.hProcess);  
free(param);  
return 0;
```